

Softwaretechnik

Prof. Dr. Raphael Herding

Vorgehensmodell

- Ermöglichen einen festgelegten organisatorischen Rahmen für die Softwareentwicklung.
- Strukturiert Softwareentwicklung in **Phasen**, **Prozesse** und **Aktivitäten** mit zugehörigen **Ergebnissen**.
- Je nach Modell können Phasen, Prozesse und Aktivitäten unterschiedlich miteinander verzahnt werden.

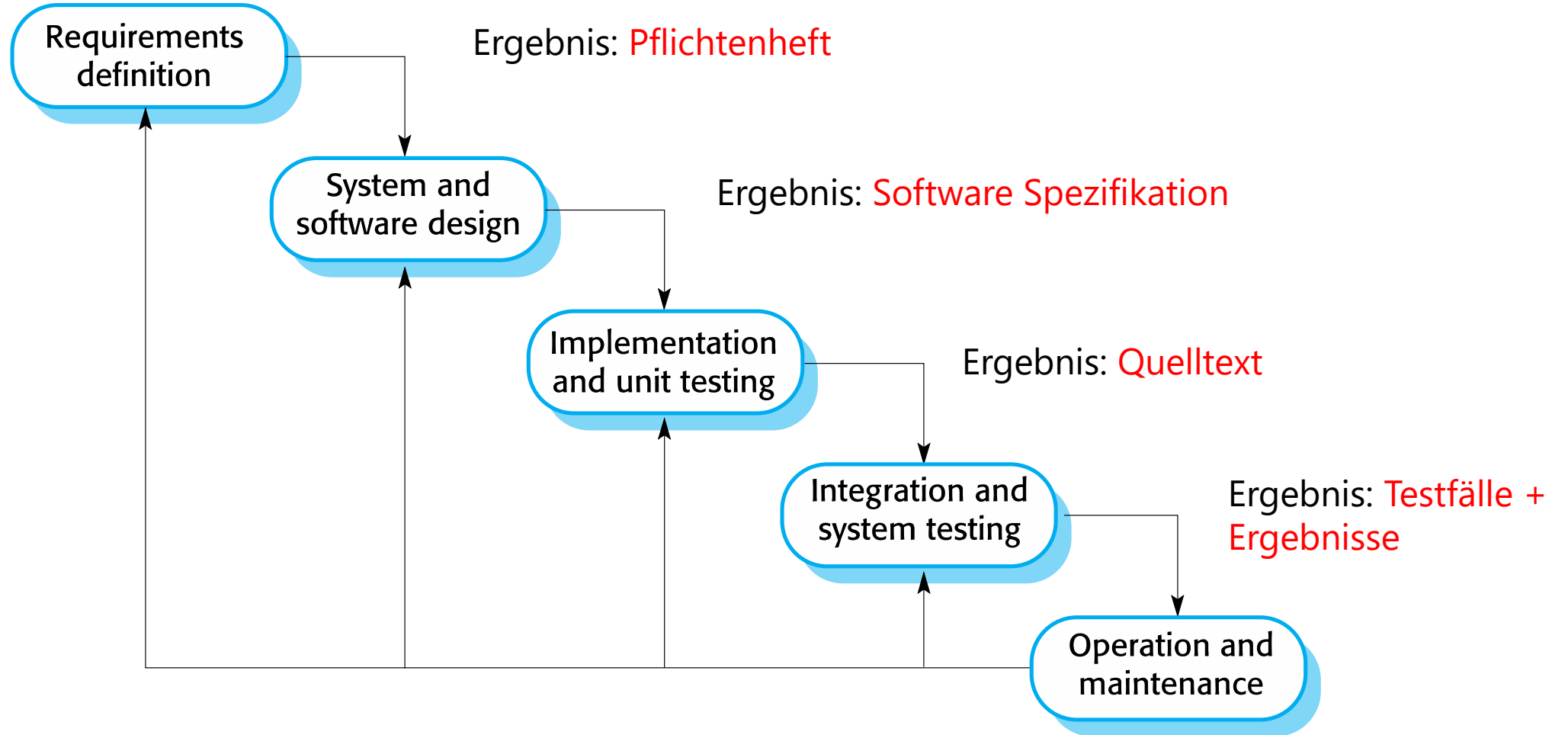
Anforderungen an „schnelle“ Softwareentwicklung

- Eine schnelle Entwicklung und Bereitstellung ist essentielles Erfolgskriterium.
- Anforderungen ändern sich schnell und es ist praktisch unmöglich, eine Reihe stabiler Softwareanforderungen zu erstellen.

Plangetriebene Entwicklung vs. Agile Entwicklung

Plangetriebene Entwicklung

Wasserfallmodell



Plangetriebene Entwicklung

Wasserfallmodell

- Im Wasserfallmodell werden verschiedene Phasen unterschieden:
 - Anforderungsanalyse und -definition
 - System- und Softwareentwurf
 - Implementierung und Einzeltests
 - Integration und Systemtests
 - Betrieb und Wartung
- Jede Phase hat entsprechende Ergebnisse, die häufig Dokumente sind.
- Nachteile
 - Berücksichtigung von Änderungen nachdem der Prozess bereits angelaufen ist
 - Im Prinzip muss eine Phase abgeschlossen sein, bevor man zur nächsten Phase übergehen kann

Plangetriebene Entwicklung

Wasserfallmodell

- Strikte sequentielle Abfolge der Einzelphasen ist unrealistisch.
- Keine Berücksichtigung von Änderungen nachdem der Prozess bereits angelaufen ist.
- Kundenwünsche (Änderungen) sind nur schwer nachträglich zu berücksichtigen.
- Ergebnis der ersten Phase (Pflichtenheft) ist bei großen Systemen nicht vollständig erfassbar.
- „Dokumentenlastig“ und „dokumentengetrieben“
- Ein lauffähiges System entsteht erst ganz am Ende der Entwicklung; häufig zu späte Rückkopplung durch den Auftraggeber.
- Qualitätssicherung findet primär gegen Ende der Entwicklung statt (Zeitdruck führt häufig zu ungenügend getesteten Programmsystemen).
- Time-to-Market-Problem (bei tendenziell langen Entwicklungszeiten)

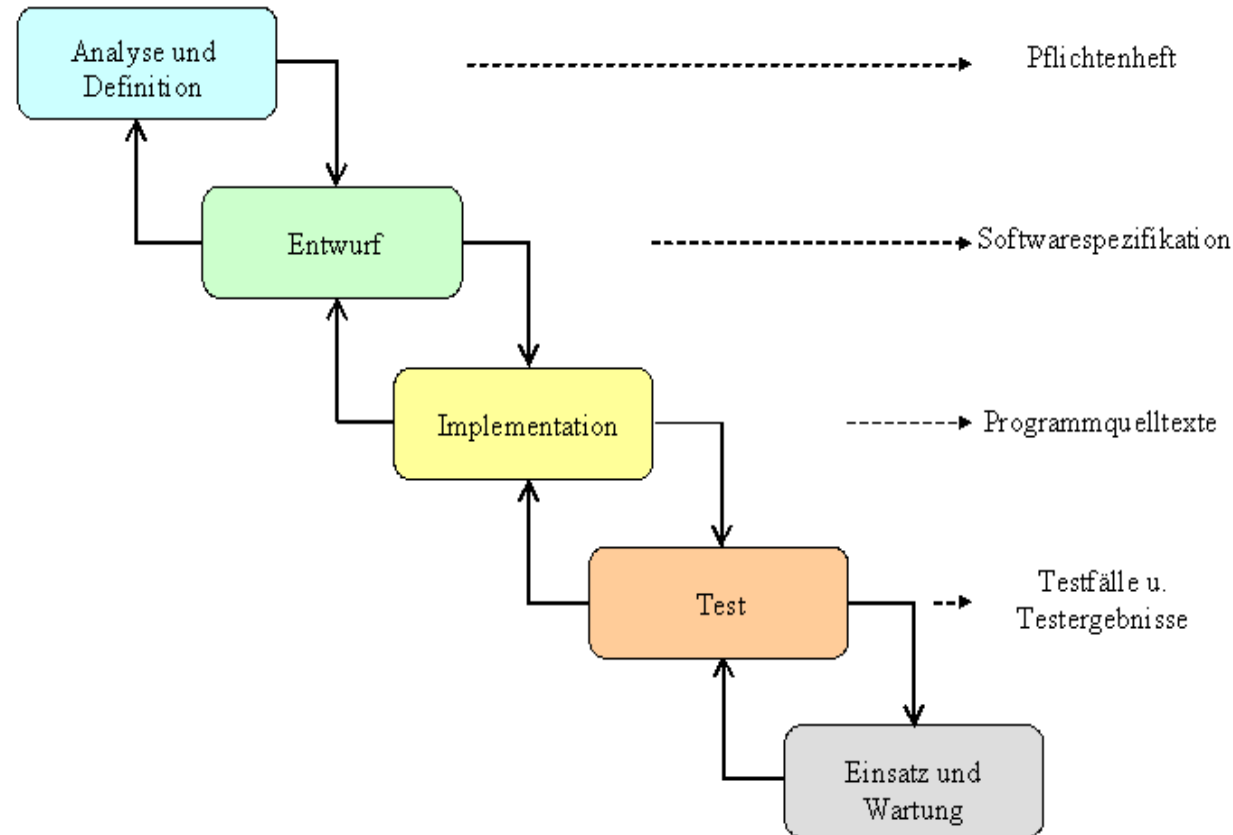
Plangetriebene Entwicklung

Wasserfallmodell

- Modell nur geeignet, wenn die **Anforderungen gut bekannt sind** und sich die Änderungen während des Entwurfsprozesses in Grenzen halten.
- Das Wasserfallmodell wird meist für **große** Systementwicklungsprojekte verwendet, bei denen ein System standortübergreifend entwickelt wird (bessere Planbarkeit).
- Bei standortübergreifender Entwicklung hilft die **Planorientierung** des Wasserfallmodells bei der Koordinierung der Arbeit.

Plangetriebene Entwicklung

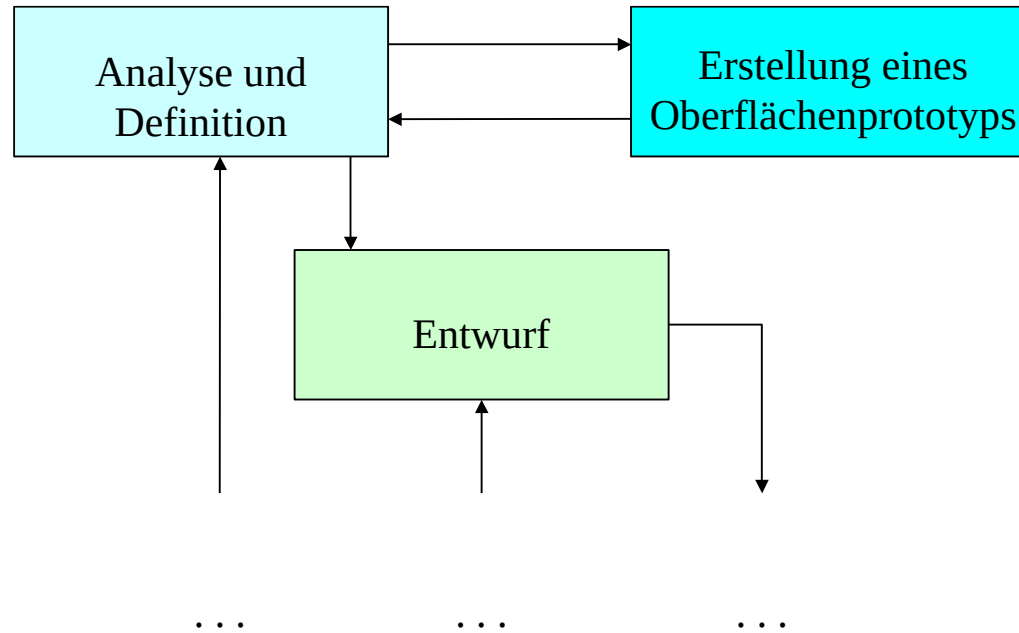
Iteriertes Phasenmodell



- hebt strikte Trennung der Phasen auf
- erlaubt Rücksprünge und Iterationen

Plangetriebene Entwicklung

Iteriertes Phasenmodell mit Prototypen



- Will Anforderungen präzisieren, indem Auftraggeber und Benutzer mit einem Prototypen (häufig Benutzungsoberfläche) konfrontiert werden.
- Prototyp kann je nach **Reifegrad behalten** oder **weggeworfen** werden.

Plangetriebene Entwicklung

Iteriertes Phasenmodell mit Prototypen

- Es gibt unterschiedliche Arten von Prototypen
 - **Oberflächenprototyp**
 - Zeigt vereinfacht die Oberfläche und hilft bei der Vorstellung des Produkts.
 - Da häufig gemacht: gute Toolunterstützung vorhanden.
 - **Architekturprototyp** → **Proof-Of-Concept (PoC)**
 - Überprüfung wichtiger Systemeigenschaften
 - dient der Risikominimierung
- Vorteile
 - Potenzielle Probleme können frühzeitig identifiziert werden.
 - Lösungsmöglichkeiten aus dem Prototypen können weitere Vorgaben abgeleitet.
 - Ermöglicht schnelles erstes Projektergebnis und minimiert Unsicherheiten/Risiken.

Plangetriebene Entwicklung

weitere Modelle

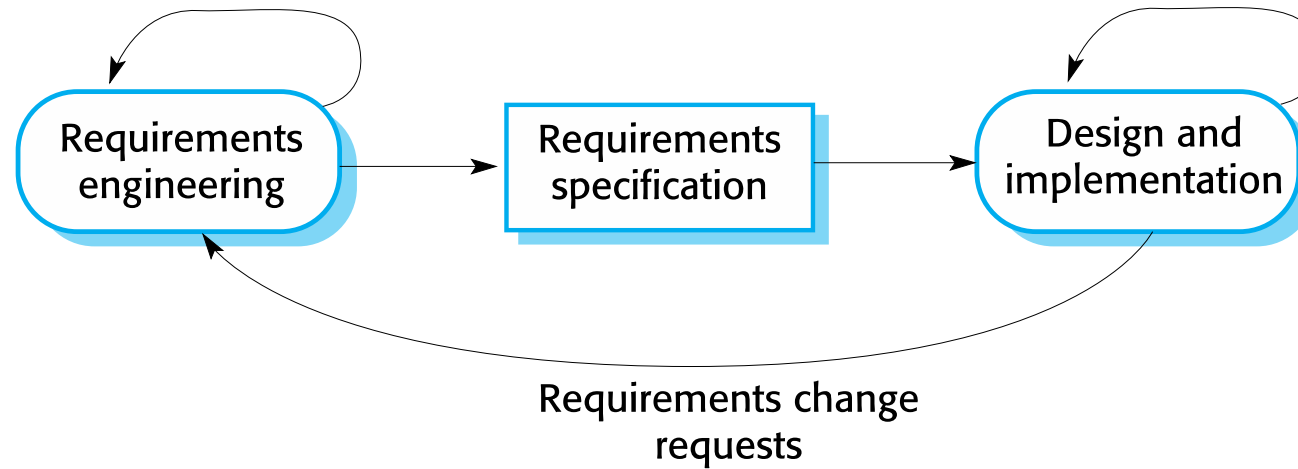
- Es existieren eine Reihe weiterer Modelle und Verfahren zur Softwareentwicklung
 - V-Modell
 - V-Model XT
 - Evolutionäre Softwareentwicklung
 - Spiralmodell
 - Transformationelle Softwareentwicklung
 - ... und viele weitere ...
- Kritik an klassischen Vorgehensmodellen
 - stark dokumentengetrieben
 - einige Modelle haben nur wissenschaftlichen Charakter und sind schwer zu übertragen
 - Prozessbeschreibungen hemmen Kreativität
 - Anpassung an neue Randbedingungen und Technologien sind aufwändig
- **Alternativer Ansatz: „Menschen machen Projekte erfolgreich, traue den Menschen“**
- **Agile Ansätze werden durch Menschen erfolgreich**

Agile Entwicklung – Eigenschaften und Ziele

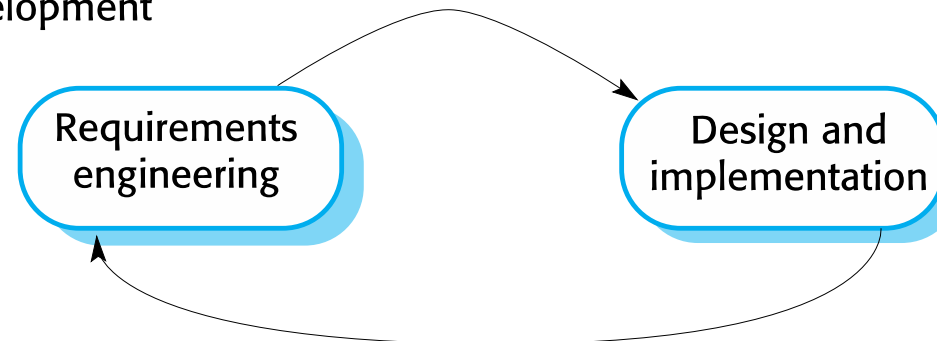
- Programmspezifikation, -entwurf und -implementierung sind miteinander verknüpft.
- Das System wird in einer Reihe von Versionen oder Inkrementen entwickelt.
- Kunde ist in Versionsdefinition und an Entwicklung beteiligt.
- Feedback-driven (Feedback durch Nutzer und Kunden; Feed-Forward durch Beratung)
- Häufige Lieferung neuer Versionen zur Bewertung durch den Kunden.
- Umfassende Werkzeugunterstützung (z. B. automatische Testwerkzeuge)
- **Ziel ist es, den Overhead zu reduzieren und schnell und mit wenig Änderungsaufwand auf veränderte Geschäftsanforderungen reagieren zu können.**

Plangetriebene und Agile Entwicklung

Plan-based development



Agile development



Agile Entwicklung – Agile Manifest

Agiles Manifest (Februar 2001)

We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland, Dave Thomas
www.agileAlliance.org

Agile Entwicklung - Einsatzbereich

- Entwicklung eines kleinen oder mittleren Produkts.
- Praktisch alle Softwareprodukte und -anwendungen werden heute nach einem agilen Ansatz entwickelt.
- Bereitschaft des Kunden aktiv an der Entwicklung mitzuwirken.

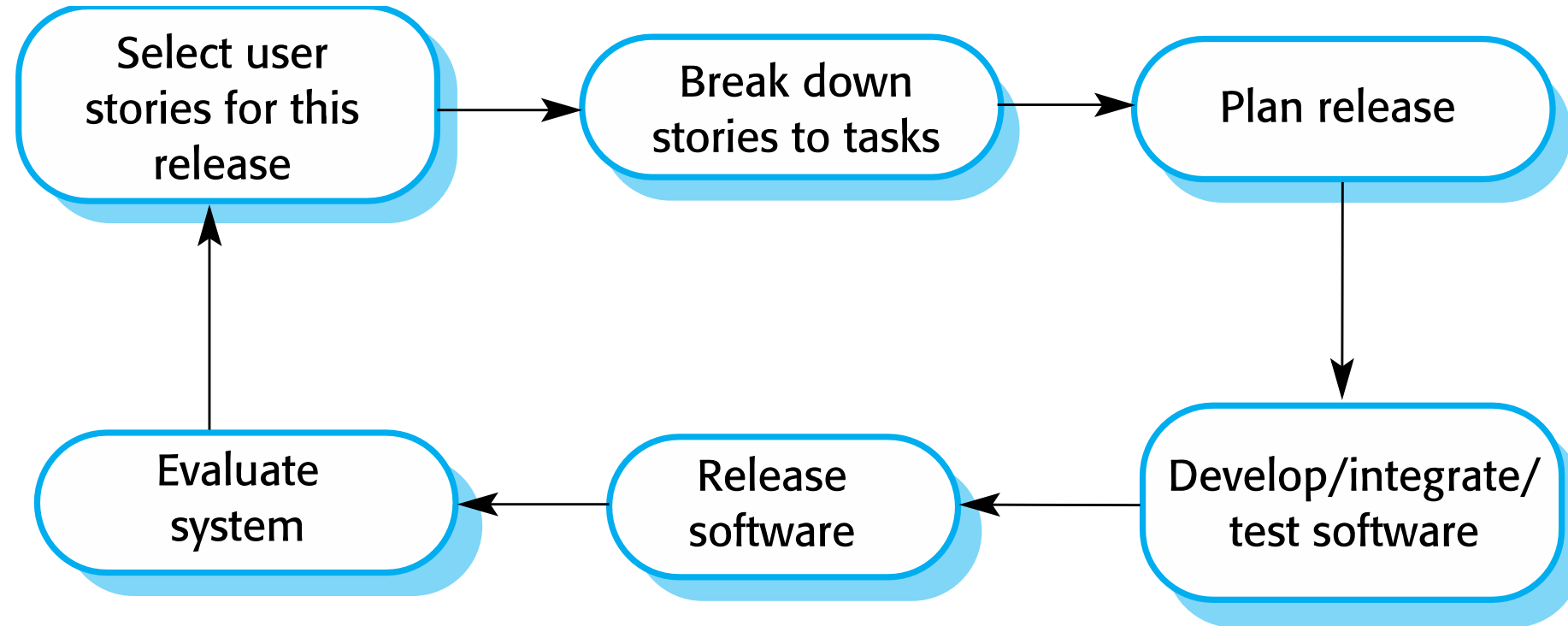
Agile Entwicklung

Extreme Programming (XP)

- Äußerst flexibel
- Kundenänderungen können sofort berücksichtigt werden
- Verzicht auf umfassende Dokumentation (leichtgewichtig)
- Kleine Projektteams (≤ 10 -15 Entwickler)
- Einige Grundprinzipien:
 - neue Versionen können mehrmals am Tag erstellt werden.
 - alle 2 Wochen Austausch und Feedback mit Kunden.
 - testfallgetriebene Entwicklung (Test-First-Ansatz).
 - bei jedem Build müssen alle Tests durchgeführt werden.
 - so einfach wie nur irgend möglich (regelmäßiges Refactoring).
 - "Kunde vor Ort" als Teil des Teams ("Story Cards" zur Funktionalität).
 - Pair-Programming.

Agile Entwicklung

Extreme Programming (XP) Release Cycle



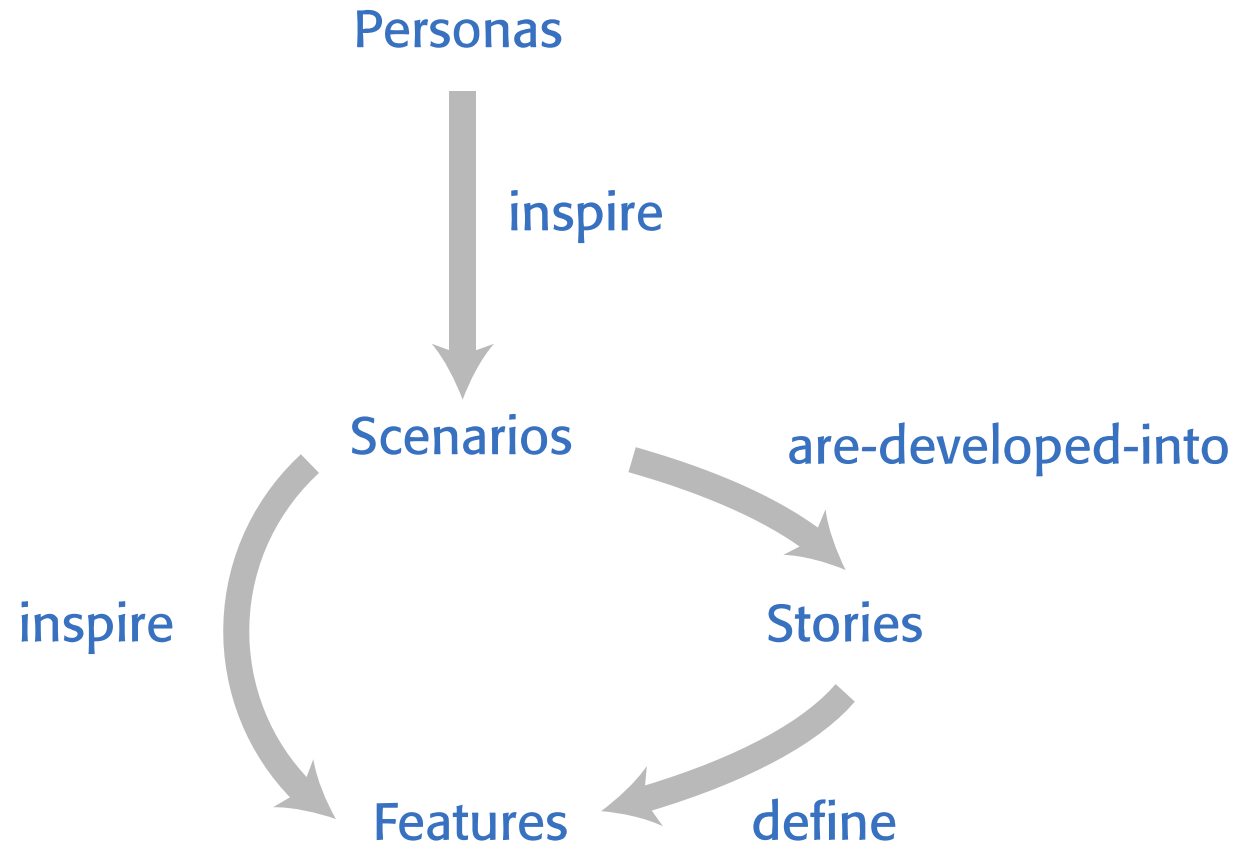
Agile Entwicklung

Extreme Programming (XP)

- Extreme Programming hat einen technischen Schwerpunkt und lässt sich nur schwer mit gelebter Managementpraxis verbinden.
- Folglich werden bei der **in der Praxis gelebten agilen Entwicklung** zwar Praktiken aus XP verwendet, aber die ursprünglich definierte Methodik ist heute nicht weit verbreitet.
- Wichtigste Praktiken aus XP
 - User-Stories für die Spezifikation
 - Refactoring
 - Test-First-Development
 - Pair-Programming

Agile Entwicklung

Systematik der Anforderungserfassung



Agile Entwicklung

Die Persona

- Um das Produkt kundenspezifisch „richtig“ zu entwickeln, muss ein gutes Verständnis über den Endnutzer hergestellt werden.
- Personas sind "imaginäre Benutzer", bei denen Sie ein Charakterporträt eines Benutzertyps ihres Produkts erstellen.
- Es ist darauf zu achten konkrete Personen und keine Rollen zu verwenden.

Agile Entwicklung

Die Persona

Merkmal	Beschreibung
Name	Dr. Anna Schneider
Rolle / Position	Fachärztin für Psychiatrie und Psychotherapie
Alter	42 Jahre
Ziele	- Patienteninformationen schnell erfassen und aktualisieren - Medikamente und Behandlungspläne dokumentieren - Termine für ambulante Konsultationen effizient organisieren
Herausforderungen	- Hoher Zeitdruck durch viele Patienten - Jederzeit aktuelle Informationen benötigen - Datenschutz bei sensiblen Patientendaten sicherstellen
Erwartungen	- Intuitive und leicht bedienbare Oberfläche - Schneller Zugriff auf Patientenakten - Sichere und DSGVO-konforme Datenübertragung

Agile Entwicklung

Die Persona

Merkmal	Beschreibung
Name	Thomas Berger
Rolle / Position	Pflegefachkraft in der psychiatrischen Station
Alter	35 Jahre
Ziele	- Rasch auf aktuelle Patientendaten (Medikamente, Anordnungen) zugreifen - Änderungen bei Vitalwerten oder Medikation direkt dokumentieren - Übergaben zwischen Schichten effizient gestalten
Herausforderungen	- Häufig wechselnde Patientenzustände erfordern schnelle Dokumentation - Schichtarbeit mit begrenzter Zeit für ausführliche Einträge - Technische Systeme müssen auch unter Zeitdruck stabil laufen
Erwartungen	- Einfache, übersichtliche Eingabemasken für schnelle Dokumentation - Offline-Zugriff auf Patientendaten bei Netzproblemen - Automatische Synchronisation nach Wiederherstellung der Verbindung

Agile Entwicklung

Szenarien

- Personas werden dazu genutzt, um Szenarien abzuleiten.
- Ein Szenario ist eine Erzählung, die beschreibt, wie ein Benutzer oder eine Gruppe von Benutzern das System verwendet.
- Es ist eine **Beschreibung einer Situation**, in der ein **Benutzer die Funktionen des Produkts nutzt**.

Agile Entwicklung

Szenarien

Aspekt	Beschreibung
Ausgangslage	Es ist Montagmorgen. Dr. Anna Schneider hat einen vollen Terminplan mit ambulanten Konsultationen.
Auslöser	Ein neuer Patient wird in der psychiatrischen Ambulanz aufgenommen. Dr. Schneider muss vor der ersten Konsultation alle relevanten Daten im Mentcare-System anlegen.
Ablauf	1. Dr. Schneider meldet sich im Mentcare-System an. 2. Sie öffnet den Bereich „Patient registrieren“ und gibt die Stammdaten (Name, Geburtsdatum, Kontaktdaten) ein. 3. Sie ergänzt bekannte Vorerkrankungen und aktuelle Medikationen. 4. Mentcare validiert die Daten und legt automatisch eine neue Patienten-ID an. 5. Dr. Schneider plant direkt im System einen Ersttermin für die Diagnostik und erstellt den ersten Behandlungsplan.

Agile Entwicklung

User Stories

- User-Stories werden häufig von Szenarien abgeleitet.
- User-Stories detaillieren Szenarien in kleine logisch zusammenhängende Teile.
- Kunde oder Benutzer ist Teil des Teams und ist für die Erstellung verantwortlich.
- Die **Nutzeranforderungen** werden in Form von User-Stories durch den Kunden formuliert.
- Diese werden vom **Entwicklungsteam** in Implementierungsaufgaben (**Tasks**) aufgeteilt.
- Die erstellten Stories und deren Tasks bilden die Grundlage für Zeit- und Kostenschätzungen.
- Der Kunde priorisiert basierend auf den Stories und definiert somit den Umfang des nächsten Releases.

Agile Entwicklung

User Stories (2)

- Zu einer User-Story gehören
 1. Ein Einleitungssatz mit Zieldefinition
 2. Aussagekräftige Beschreibung der Story mit Eigenschaften
 3. Definition of Done (wann ist die Story als erfüllt anzusehen)
- 1. Einleitung mit Zieldefinition
 - Eine User-Story wird stets im folgenden Format formuliert:
 - **As a** <role>, I <want | need> **to** <do something>
 - (Bsp. As a teacher, I want to tell all members of my group when new information is available)
 - Als Alternative kann auch eine Begründung eingefügt werden:
 - **As a** <role> I <want | need> **to** <do something> **so that** <reason>
 - (Bsp. As a teacher, I need to be able to report who is attending a class trip so that the school maintains the required health and safety records.)
 - In deutsch jeweils "Als [Rolle] [möchte | benötige] ich <Grund | Irgendetwas>, [damit]."

Agile Entwicklung

User Stories (3)

- Zu einer User-Story gehören
 1. Ein Einleitungssatz mit Zieldefinition
 2. Aussagekräftige Beschreibung der Story mit Eigenschaften
 3. Definition of Done (wann ist die Story als erfüllt anzusehen)
- 2. Häufig wird Freitext verwendet
- 3. Definition of Done
 - Gibt an, wann eine User-Story als erledigt anzusehen ist.
 - Die Story darf erst dann in den Zustand „abgeschlossen“ versetzt werden, wenn die Definition of Done nachweislich erfüllt wurde.
 - Dazu werden typischerweise Tests erstellt, die die Definition of Done automatisiert prüfen.

Agile Entwicklung

User Stories

User Story 1: Patientendaten erfassen

Aspekt	Beschreibung
Einleitungssatz (Zieldefinition)	Als Fachärztin für Psychiatrie möchte ich neue Patienten im Mentcare-System anlegen , damit ich alle relevanten Daten für Diagnose und Behandlung sofort verfügbar habe .
Beschreibung	Ausschnitt! - Dr. Anna Schneider kann im Modul „Patient registrieren“ alle notwendigen Stammdaten eingeben (Name, Geburtsdatum, Kontaktdaten). - Vorerkrankungen und aktuelle Medikationen können direkt ergänzt werden. - Das System validiert Eingaben auf Vollständigkeit und Plausibilität. - Bei fehlerhaften Eingaben werden klare Fehlermeldungen angezeigt.
Definition of Done	- Der Patient ist mit einer eindeutigen Patienten-ID im System erfasst. - Alle relevanten Stammdaten sowie Vorerkrankungen/Medikationen sind gespeichert. - Bei Testläufen sind keine Validierungsfehler vorhanden.

Agile Entwicklung

User Stories

User Story 2: Termin für Erstdiagnose planen

Aspekt	Beschreibung
Einleitungssatz (Zieldefinition)	Als Fachärztin für Psychiatrie möchte ich direkt nach der Registrierung einen Ersttermin anlegen , damit Patient und Team schnell über den geplanten Diagnosetermin informiert sind .
Beschreibung	Ausschnitt! - Nach erfolgreicher Registrierung öffnet sich automatisch das Modul „Terminplanung“. - Dr. Anna Schneider kann Datum, Uhrzeit und behandelnden Arzt auswählen. - Das System prüft auf Terminkonflikte und zeigt freie Zeitfenster an. - Der Termin wird automatisch im Patientenprofil hinterlegt und eine Bestätigung wird an die Verwaltung gesendet.
Definition of Done	- Ein Ersttermin ist im System erstellt und im Patientenprofil sichtbar. - Terminbestätigung wurde automatisch generiert und an Verwaltung/Patient weitergegeben. - Bei Testläufen sind keine Terminüberschneidungen aufgetreten.

Agile Entwicklung

Task

- Tasks werden häufig aus User-Stories abgeleitet.
- Tasks detaillieren User-Stories in kleine implementierbare Einheiten.
- Manchmal können User-Stories auch direkt als Tasks interpretiert werden.
- Ein Task sollte **ein bis drei Tage Realisierungsaufwand** nicht übersteigen.

Agile Entwicklung

Task

Task 1: Frontend – Eingabemaske „Patient registrieren“

Bezug zu User Story 1: Erlaubt Dr. Schneider, alle relevanten Stammdaten, Vorerkrankungen und Medikationen einzugeben.

UI/UX-Anforderungen

- Seite/Modal: `/patients/new` mit primärer Aktion „Speichern“ und Sekundäraktion „Abbrechen“.
- Formular-Gruppen: **Stammdaten**, **Kontakt**, **Medizinisch**, **Einverständnis/DSGVO**.
- Live-Validierung pro Feld (onBlur) + Gesamtsumme beim Submit (scroll to first error).
- Erfolgs-Toast („Patient angelegt“) und Redirect auf `/patients/{patientId}`.

Felder & Typen (Client)

- Stammdaten:
 - `Vorname` (String, Pflicht, 1–50), `Nachname` (String, Pflicht, 1–50)
 - `Geburtsdatum` (Date, Pflicht, ISO-8601, darf nicht in der Zukunft liegen)
 - `Geschlecht` (Enum: {„weiblich“, „männlich“, „divers“, „unbekannt“}, optional)
- Kontakt:
 - `Adresse` (String, optional, max. 200), `Telefonnummer` (String, optional, E.164 empfohlen), `Email` (String, optional, RFC 5322)
- Medizinisch:
 - `Vorerkrankungen` (Mehrfachauswahl/Tags, optional)
 - `AktuelleMedikation` (Tabelle: Name:String, Dosis:String, Häufigkeit:String; 0..n)

- Rechtliches:
 - `EinwilligungDatenverarbeitung` (Checkbox, Pflicht)
 - `Notfallkontakt` (optional: Name, Beziehung, Telefon)

Validierungsregeln (Client)

- Pflichtfelder: Vorname, Nachname, Geburtsdatum, Einwilligung.
- `Geburtsdatum` ≤ heute.
- `Email` gültig, `Telefonnummer` nur Ziffern/„+“, max. 20.

Akzeptanzkriterien

- Bei gültiger Eingabe wird **POST /api/patients** abgesetzt; nach **201** Redirect auf Detailseite.
- Bei Validierungsfehlern: Fehler-Tooltip am Feld + Zusammenfassung oben.
- Formular ist per Tastatur bedienbar (A11y: Labels, ARIA, Fokusmanagement).

Agile Entwicklung

Pair-Programming

- Programmierer sitzen gemeinsam an einem Computer und entwickeln Software.
- Die Paare werden dynamisch gebildet (stärkt Teamzusammenhalt).
- Wissensaustausch während des Pair-Programmings ist essentiell.
- Pair-Programming kann individuellen Skill verbessern.
- „Kopfmonopole“ werden verringert.
- Risiko bei Weggang oder Krankheit wird minimiert.

Agiles Projektmanagement

- Projektmanager müssen das Projekt so managen, dass die Software rechtzeitig und innerhalb des geplanten Budgets fertiggestellt wird.
- **Standardansatz: Plangetriebene Entwicklung**
- Manager erstellen einen Plan für das Projekt (was soll wann, wie geliefert werden).
- **Agiles Projektmanagement** erfordert einen **anderen Ansatz**, der an die inkrementelle Entwicklung und die in den agilen Methoden verwendeten Praktiken angepasst ist.

Scrum als agilen Planungsansatz

- Scrum ist eine agile Methode, die sich auf das **Management** der **iterativen Entwicklung** und **nicht auf spezifische agile Praktiken** konzentriert.
- Es gibt drei Phasen in Scrum:
 1. **Initiale Planungsphase** - Festlegung der allgemeinen Projektziele. Grober Entwurf der Softwarearchitektur.
 2. **Sprint-Zyklen Phase** - jeder Zyklus liefert ein Inkrement (neue Version) des Systems.
 3. **Projektabschlussphase** - Projekt wird abgeschlossen und wie zu Beginn spezifiziert übergeben. Es erfolgt eine **Reflexion** und es werden **gezogenen Lehren** bewertet.

Scrum

Eigenschaften und Ablauf

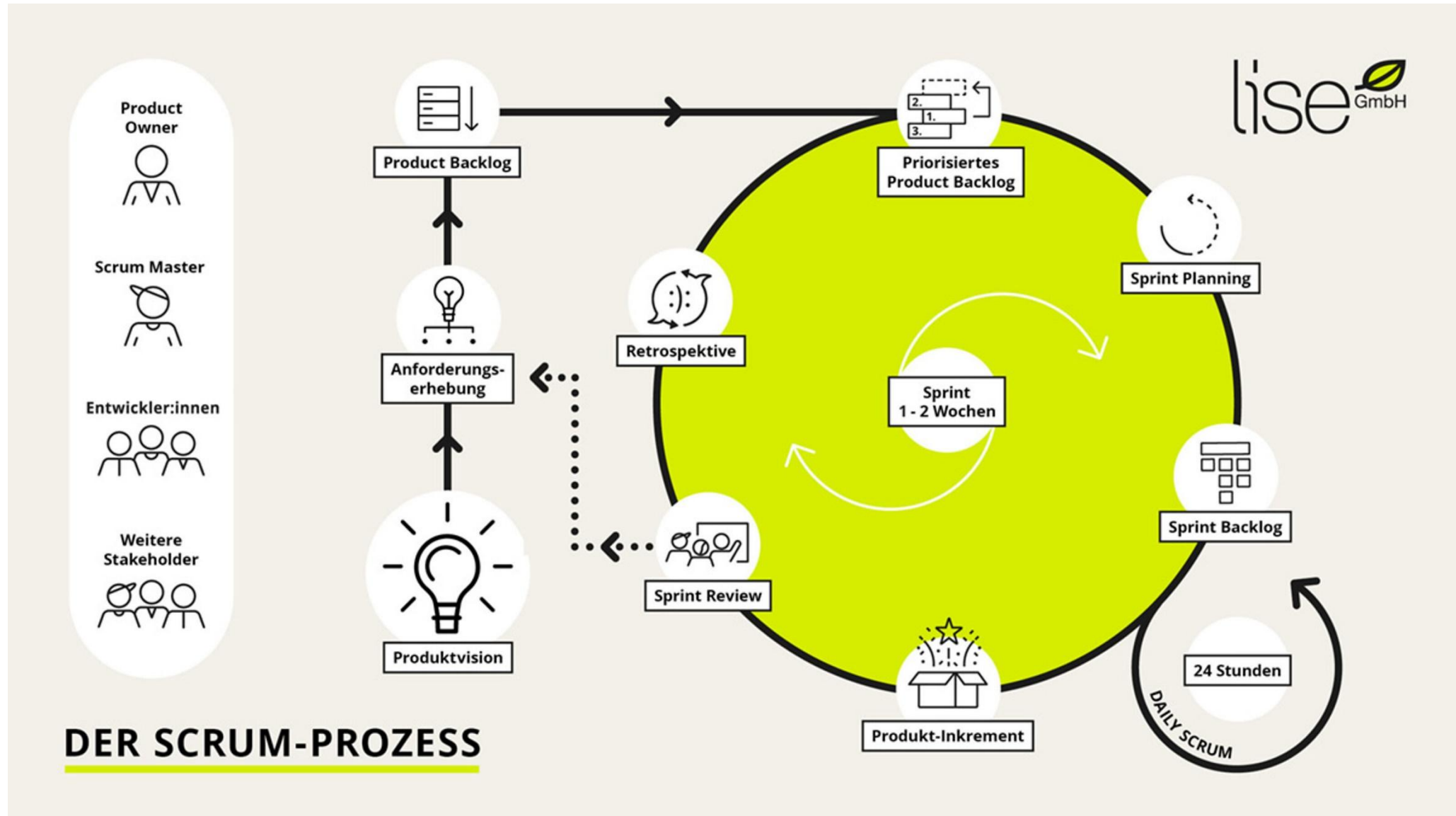
- „Scrum“: Gedränge beim Rugby
- Scrum Teams bestehen aus 5-10 Personen mit unterschiedlichen Fähigkeiten:
 - Koordinator (Scrum Master), Anforderungsdefinerer (Product Owner), Entwickler-Team, ...
- Bei größeren Projekten werden verschiedene Teams durch einen Scrum Master koordiniert.
- Zentrales Steuerungselement: Scrum Meetings; jeden Tag maximal 15 Minuten:
 - was habe ich seit letztem Meeting gemacht?
 - was werde ich bis zum nächsten Meeting machen?
 - was hat meine Arbeit behindert?
- Scrum Master ist Koordinator; beseitigt Behinderungen; kommuniziert im Unternehmen
- **Arbeitsablauf** im Team **wird vom Team selbst gesteuert.**

Scrum

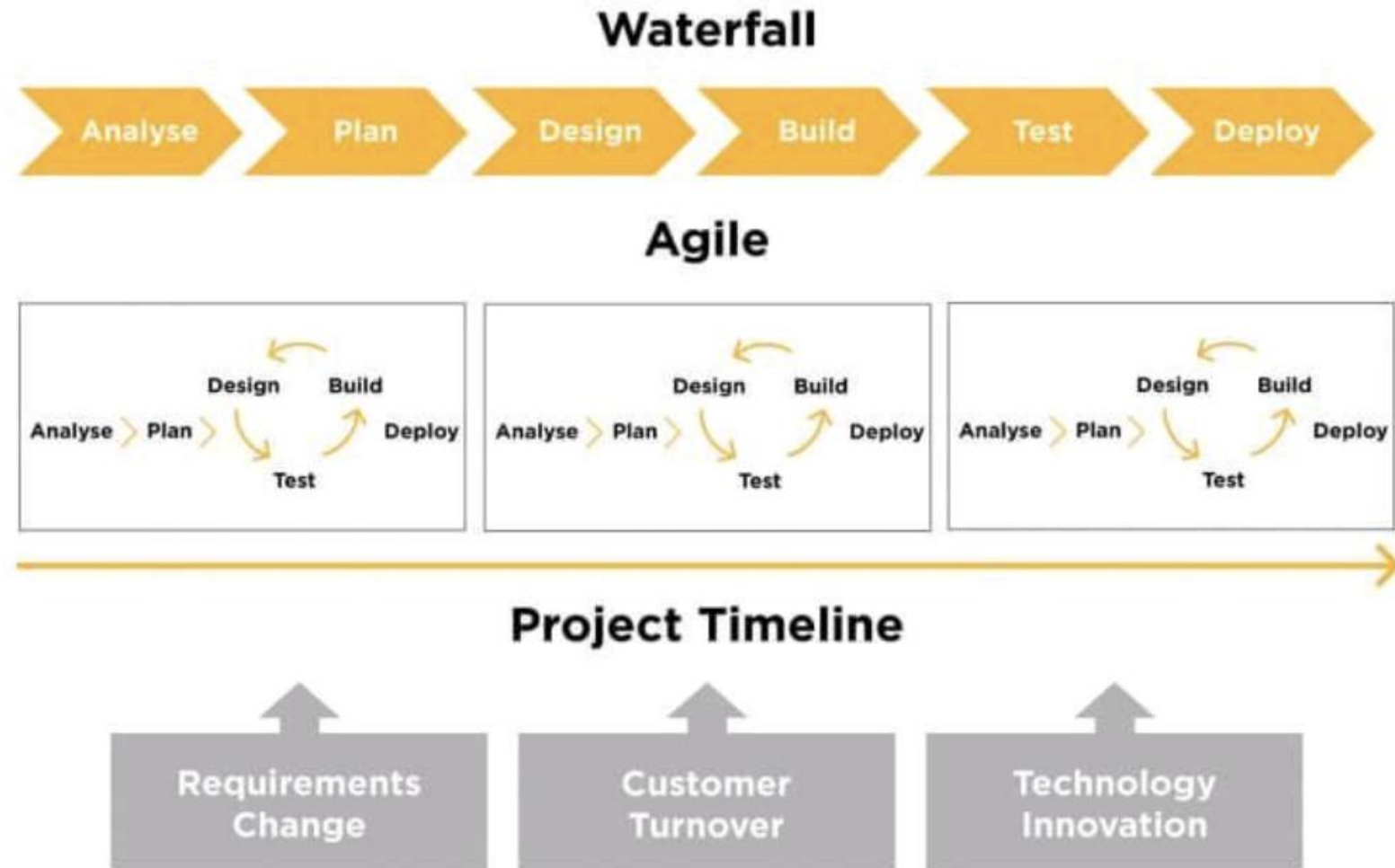
Eigenschaften und Ablauf

- **Projektplanung:** Eng mit dem Kunden werden Hauptaufgaben identifiziert (Stack mit **Product Backlog**).
- **Product Owner** nimmt Rolle des Kunden ein (kommt vom Kunden; oder erfahren im Anwendungsgebiet).
- **Scrum Team schätzt Aufwände:** wählt mit Kunden aus **Product Backlog** wichtigste nächste Aufgaben für kommende Iteration (**Sprint**) aus (hinzufügen in **Sprint Backlog**).
- Jeder Sprint dauert ca. 2-4 Wochen
 - Basis ist Sprint Backlog mit priorisierten Aufgaben
 - sorgt für **nicht unterbrochene Arbeitsphase** des Teams (keine Störung durch Kunden)
- Nach jedem Sprint erfolgt Analyse und Feedback mit dem Kunden
 - Was wurde erreicht?
 - Wie kann Projekt verbessert werden?
 - Sind Anforderungen noch aktuell?

Scrum Prozess



Plangetriebene und agile Entwicklung



Scrum

Wann ist SCRUM eigentlich sinnvoll einzusetzen?



Scrum eignet sich für Teams, welche - zum einen - regelmäßig innovativ und kreativ Produkte entwickeln wollen und dabei viele parallele Aufgaben bewältigen müssen, die jedoch alle auf ein großes Ganzes einzahlen und dies - zum anderen - in einem komplexen Umfeld tun.

Scrum

Wann ist SCRUM eigentlich sinnvoll einzusetzen?

Komplexes Umfeld?

Ein komplexes Umfeld ist ein Umfeld, was geprägt ist von Herausforderungen bei denen Ursache und Wirkung im Vorfeld nicht mehr klar zuzuordnen sind. Vereinfacht gesagt: Wenn ich A tue, kann ich nicht davon ausgehen, dass B dabei raus kommt, sondern eben etwas Unbekanntes - wie D oder Z.

Scrum

Wann ist SCRUM eigentlich sinnvoll einzusetzen?

Komplexes Umfeld?

Ein komplexes Umfeld ist ein Umfeld, was geprägt ist von Herausforderungen bei denen Ursache und Wirkung im Vorfeld nicht mehr klar zuzuordnen sind. Vereinfacht gesagt: Wenn ich A tue, kann ich nicht davon ausgehen, dass B dabei raus kommt, sondern eben etwas Unbekanntes - wie D oder Z.



Der einzige Weg, dies herauszufinden ist, daher etwas auszuprobieren, dann zu erkennen, was passiert ist (Ist meine Hypothese eingetroffen oder nicht?) und dann darauf zu reagieren und beispielsweise Aspekte zu ändern. Und dann das Ganze zu wiederholen.

Scrum

Wann ist SCRUM eigentlich sinnvoll einzusetzen?

Komplexes Umfeld?

Ein komplexes Umfeld, Herausforderungen, Ursache und Wirkung nicht mehr vereinfacht darstellen kann ich mir, dass B dabei eben ein wie D oder Z.

Wenn ich als Team also nicht über einen längeren Zeitraum funktionierend planen kann, kann mir ein zyklisches Arbeiten mit Scrum helfen, viel besser auf das reagieren zu können, was kommt und bisher nicht absehbar war.



herauszufinden bieren, dann ist getroffen darauf zu eise Aspekte s Ganze zu

wiedernoten.

Scrum

Vorteile

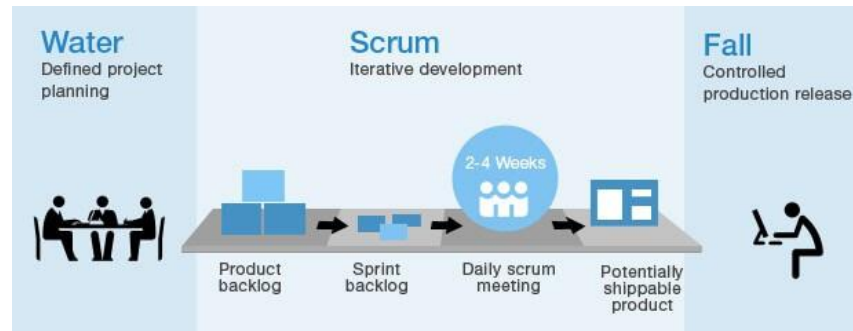
- Das Produkt wird in überschaubare und besser verständliche Teile zerlegt.
- Unbeständige Anforderungen können den Entwicklungsfortschritt nicht stören.
- Das gesamte Team hat einen Überblick über alles.
- Die Kunden sehen, dass die Inkremente pünktlich geliefert werden (Vertrauen), und erhalten frühzeitig Rückmeldungen darüber, wie und ob das Produkt funktioniert.
- Das Vertrauen zwischen Kunden und Entwicklern wird gestärkt und es entsteht eine positive Kultur, in der Produktzugehörigkeit entwickelt wird.

Wartung bei agiler Softwareentwicklung

- Die Hauptprobleme sind
 - Mangel an Produktdokumentation,
 - Einbindung der Kunden in den Entwicklungsprozess sowie die
 - Aufrechterhaltung der Kontinuität des Entwicklungsteams.
- Um die Probleme zu bewältigen werden Strategien wie DevOps (Development and Operations) eingesetzt.
- Entwicklungsteam ist für den Betrieb verantwortlich (**You build it, You own it**).
- Nicht immer möglich (Arbeitsrecht, Verfügbarkeit, Kultur, Wille, Skill,).

Wartung bei agiler Softwareentwicklung

- Bei Systemen mit langer Lebensdauer ist DevOps häufig ein Problem, da die ursprünglichen Entwickler zumeist nicht dauerhaft verfügbar sind.
- In der Praxis werden daher agile Methoden häufig mit planungsgetriebenen kombiniert
- Water-Scrum-Fall: Mischung aus Scrum und Wasserfall



- Planungsgetriebener Rahmen und agile Entwicklung mit explizierter Berücksichtigung langfristiger Wartung (Controlled Production Release).

Auswahlhilfe



Zusammenfassung

- Es existieren zahlreiche Vorgehensmodelle mit spezifischen Stärken und Schwächen.
- Findung eines Kompromisses zwischen geforderter Standardisierung und notwendigem Freiraum zur Kreativität nötig.
- Vereinbartes Vorgehensmodell sollte von Entwicklern und Management als **zielführend** und **hilfreich** empfunden werden.
- **Aktuell modern:** „leichtgewichtige, agile Vorgehensmodelle“ im Gegensatz zur Software „Bürokratie“ „schwergewichtiger“ Vorgehensmodelle.